

Primer Parcial Práctico

Cinemática Directa e Inversa del Robot Kinova Gen3 6-DOF con ROS 2 Jazzy

Grupo 9

Miguel Angel Torrez Valeriano

Henry Pablo Lima Poma

IMT-342 Robótica – Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—Este trabajo presenta el modelado cinemático del manipulador Kinova Gen3 de seis grados de libertad y su implementación en ROS 2 Jazzy. A partir de la geometría del modelo oficial `kortex_description`, se asignaron los marcos de referencia y se obtuvo una tabla Denavit–Hartenberg estándar, complementada con una transformación fija entre la base y el primer marco. Con ella se desarrolló la cinemática directa en forma semisimbólica, obteniendo la posición y la orientación del efector final, y el Jacobiano posicional por vía geométrica y analítica. La cinemática inversa de posición se resolvió con un método iterativo basado en la pseudoinversa del Jacobiano, con paso acotado y límites articulares. Ambos modelos se implementaron en dos nodos propios: `fk_node`, suscrito a `/joint_states`, e `ik_node`, que recibe objetivos en `/target` y publica la solución en `/joint_states`. La cinemática directa coincidió con TF de ROS en tres configuraciones, con errores menores a 0.5 mm, y la inversa convergió en tres objetivos con errores menores a 0.1 mm en 13 a 17 iteraciones.

Index Terms—Denavit–Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, manipulador serial.

I. DESCRIPCIÓN DEL ROBOT ASIGNADO

El robot asignado para el presente trabajo es el Kinova Gen3 de 6 grados de libertad, un manipulador robótico ultraligero desarrollado por Kinova Robotics para aplicaciones de investigación, automatización y manipulación robótica. El modelo Gen3 se caracteriza por emplear actuadores inteligentes con sensores de torque integrados y por permitir el control de sus articulaciones mediante la plataforma KINOVA KORTEX [1]. Para el modelado se utilizó el paquete oficial `kortex_description` [2], en su configuración Gen3 de 6 GDL sin gripper.

I-A. Datos generales

Cuadro I
DATOS PRINCIPALES DEL ROBOT

Dato	Valor
Fabricante	Kinova Robotics
Modelo	Kinova Gen3
Grados de libertad	6
Tipo de articulaciones	Revolutas
Alcance máximo	891 mm
Marco base	<code>base_link</code>
Efector final	<code>end_effector_link</code>
Unidades empleadas	SI: metros y radianes

I-B. Cadena cinemática

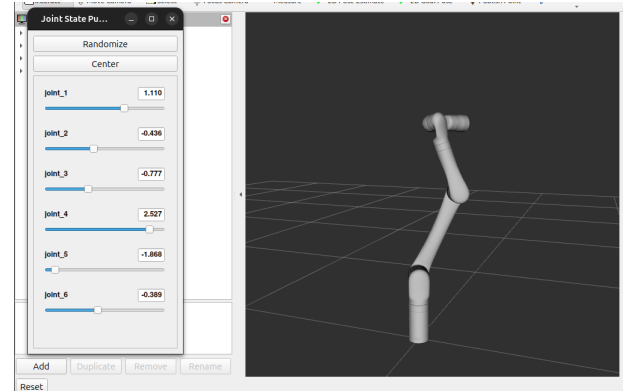


Figura 1. Robot Kinova Gen3 6-DOF en RViz2 con el Joint State Publisher GUI; los controles `joint_1` a `joint_6` corresponden a las articulaciones q_1 a q_6 .

II. ASIGNACIÓN DE MARCOS DE REFERENCIA

La asignación de marcos se realizó en la configuración cero del modelo ($q = 0$), en la que el brazo queda completamente vertical. En esta postura, los ejes de las articulaciones siguen el patrón vertical–horizontal–horizontal–vertical–horizontal–vertical: `joint_2` y `joint_3` son paralelas (hombro y codo de un brazo antropomórfico), y `joint_4`, `joint_5` y `joint_6` forman una muñeca prácticamente esférica, cuyos ejes solo se separan por desplazamientos del orden de milímetros.

Los marcos se asignaron con los siguientes criterios:

- z_{i-1} coincide con el eje de giro de la articulación i , con el mismo sentido que en el URDF.
- Cuando dos ejes consecutivos se cortan, el origen O_i se coloca en la intersección y x_i es perpendicular a ambos ejes ($a_i = 0$).
- Cuando son paralelos (joint_2 y joint_3), x_i se ubica sobre la normal común entre ellos, cuya longitud es a_2 .
- y_i se obtiene con la regla de la mano derecha.
- El marco $\{6\}$ se ubica en end_effector_link, con la misma orientación que el modelo oficial.

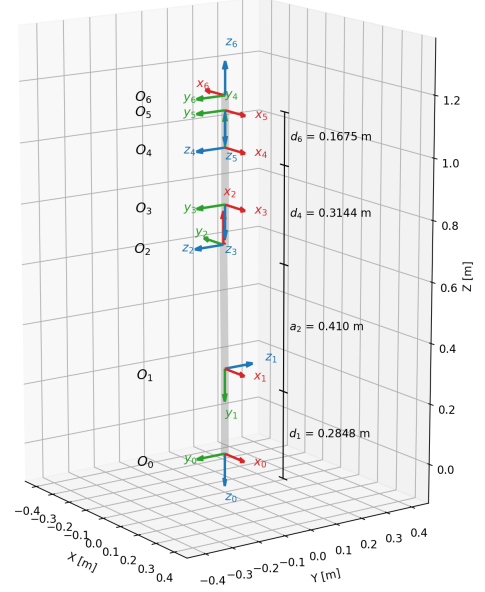
Como en el URDF el eje de joint_1 apunta en la dirección $-z$ de base_link, el marco $\{0\}$ se obtiene girando base_link un ángulo π alrededor de x . Esta transformación fija se presenta por separado en la Sección III.

La Tabla II resume el origen y la orientación de cada marco en $q = 0$, expresados en base_link, y la Fig. 2 los muestra gráficamente. La figura se generó a partir de las matrices DH del propio modelo. Los marcos $\{2\}$ – $\{3\}$ (codo) y $\{4\}$ – $\{5\}$ (centro de la muñeca) prácticamente coinciden, por lo que en la figura los marcos $\{3\}$ y $\{5\}$ se dibujaron desplazados solo para su visualización.

Cuadro II
MARCOS DE REFERENCIA EN $q = 0$, EXPRESADOS EN BASE_LINK

Marco	Origen [m]	x_i	z_i	Eje de
0	(0, 0, 0)	+X	−Z	joint_1
1	(0, 0, 0.2848)	+X	+Y	joint_2
2	(0, −0.0054, 0.6948)	+Z	−Y	joint_3
3	(0, 0.0010, 0.6948)	+X	−Z	joint_4
4	(0, 0.0010, 1.0092)	+X	−Y	joint_5
5	(0, 0.0014, 1.0092)	+X	−Z	joint_6
6	(0, 0.0014, 1.1766)	−X	+Z	efector

Asignación de frames DH — Kinova Gen3 6-DOF ($q = 0$)



Frames 3 y 5 desplazados verticalmente solo para visualización (línea punteada).
Sus orígenes reales coinciden con los de los frames 2 y 4 (diferencias < 7 mm).

Figura 2. Asignación de marcos de referencia del robot Kinova Gen3 6-DOF en $q = 0$ (x_i en rojo, y_i en verde, z_i en azul).

II-A. Convención utilizada

En este trabajo se utiliza la convención Denavit–Hartenberg estándar:

$$A_{i-1}^i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i). \quad (1)$$

Su forma matricial es:

$$A_{i-1}^i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (2)$$

donde $c_{\theta_i} = \cos \theta_i$ y $s_{\theta_i} = \sin \theta_i$.

III. PARÁMETROS DENAVIT–HARTENBERG

Los parámetros se obtuvieron a partir de la asignación de marcos de la Fig. 2, construida sobre la geometría del modelo oficial kortex_description (Gen3, 6 GDL, sin gripper). En el URDF, cada articulación gira alrededor del eje z de su propio marco, por lo que cada z_{i-1} se hizo coincidir en dirección y sentido con el eje de la articulación i . Con esta elección, todas las variables quedan de la forma $\theta_i = q_i + \text{offset}$, sin cambios de signo.

En el URDF, el eje de joint_1 apunta en la dirección $-z$ de base_link. Por ello, el marco $\{0\}$ no coincide con base_link, sino que se relaciona con él mediante una transformación fija, que se presenta por separado:

$$T_{base}^0 = R_x(\pi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

Cuadro III

PARÁMETROS DENAVIT-HARTENBERG DEL ROBOT KINOVA GEN3 6-DOF

Transf.	θ_i	d_i [m]	a_i [m]	α_i	Descripción
A_0^1	q_1	-0.28481	0	$\pi/2$	base-hombro
A_1^2	$q_2 - \pi/2$	-0.005375	0.410	π	hombro-codo
A_2^3	$q_3 - \pi/2$	-0.006375	0	$\pi/2$	codo-antebrazo
A_3^4	q_4	-0.31436	0	$-\pi/2$	antebrazo-muñeca
A_4^5	q_5	-0.00035	0	$\pi/2$	muñeca 1-2
A_5^6	$q_6 + \pi$	-0.167455	0	π	muñeca-efector

III-A. Verificación geométrica de los parámetros

Las distancias de la Tabla III provienen de los orígenes <origin xyz> de cada articulación del URDF:

- $d_1 = -(0.15643 + 0.12838) = -0.28481$ m: altura de la base al eje del hombro (joint_1 y joint_2).
- $a_2 = 0.410$ m: separación entre los ejes paralelos de joint_2 y joint_3 (brazo). Es el único a_i no nulo, porque el resto de ejes consecutivos se cortan.
- $d_4 = -(0.20843 + 0.10593) = -0.31436$ m: longitud del antebrazo, de joint_3 a joint_5.
- $d_6 = -(0.10593 + 0.061525) = -0.167455$ m: distancia del centro de la muñeca al efector (end_effector_link).
- d_2, d_3 y d_5 son desplazamientos laterales del modelo real (0.005375, 0.006375 y 2×0.000175 m). Se incluyeron para que la cinemática directa coincidiera exactamente con el modelo de ROS.

Los valores de d_i son negativos porque, al seguir el sentido de los ejes del URDF, los z_{i-1} apuntan en sentido opuesto al avance de la cadena. Los ángulos $\alpha_i = \pm\pi/2$ reflejan ejes consecutivos perpendiculares; $\alpha_2 = \pi$ refleja los ejes paralelos de sentido opuesto de joint_2 y joint_3. Los offsets $-\pi/2$ en θ_2 y θ_3 , y $+\pi$ en θ_6 , hacen que $q = 0$ corresponda a la configuración cero del URDF (brazo vertical).

IV. CINEMÁTICA DIRECTA

La pose del efector respecto de base_link se obtiene con la transformación fija (3) y el producto de las seis matrices DH:

$$T_{base}^6 = T_{base}^0 T_0^6 = R_x(\pi) A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6, \quad (4)$$

$$T_{base}^6 = \begin{bmatrix} R(q) & p(q) \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R(q) \in SO(3). \quad (5)$$

IV-A. Matrices homogéneas individuales

Sustituyendo la Tabla III en (2), con $c_i = \cos q_i$ y $s_i = \sin q_i$, se obtienen [4]:

$$A_0^1 = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

$$A_1^2 = \begin{bmatrix} s_2 & -c_2 & 0 & a_2 s_2 \\ -c_2 & -s_2 & 0 & -a_2 c_2 \\ 0 & 0 & -1 & d_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

$$A_2^3 = \begin{bmatrix} s_3 & 0 & -c_3 & 0 \\ -c_3 & 0 & -s_3 & 0 \\ 0 & 1 & 0 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

$$A_3^4 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

$$A_4^5 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (10)$$

$$A_5^6 = \begin{bmatrix} -c_6 & -s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & -1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

Los offsets de θ_2, θ_3 y θ_6 ya están incorporados: por ejemplo, $\cos(q_2 - \pi/2) = s_2$ y $\cos(q_6 + \pi) = -c_6$. La última fila de todas las matrices es $[0 \ 0 \ 0 \ 1]$.

IV-B. Matriz de transformación total

El producto (4) se desarrolló simbólicamente con SymPy a partir de las matrices anteriores. Como joint_2 y joint_3 son paralelas, sus ángulos aparecen combinados, por lo que se definen las abreviaturas:

$$c_{23} = \cos(q_2 - q_3), \quad s_{23} = \sin(q_2 - q_3). \quad (12)$$

El resultado se presenta separado en su parte de posición, $p(q)$, y de orientación, $R(q) = [r_{ij}]$, en las subsecciones siguientes. Cada elemento de T_{base}^6 queda expresado únicamente en función de $q_1, \dots, q_6$ y de los parámetros de la Tabla III.

IV-C. Posición y orientación del efector

Para expresar la posición de forma compacta se definen dos términos auxiliares: la proyección radial ρ del efector sobre el plano del brazo y el desplazamiento lateral h :

$$\rho = a_2 s_2 - d_4 s_{23} + d_6 (c_4 c_{23} s_5 - c_5 s_{23}) - d_5 c_{23} s_4, \quad (13)$$

$$h = d_2 - d_3 - d_5 c_4 - d_6 s_4 s_5. \quad (14)$$

Con ellos, la posición del efector es:

V. JACOBIANO DEL MANIPULADOR

$$p_x = c_1 \rho + s_1 h, \quad (15)$$

$$p_y = -s_1 \rho + c_1 h, \quad (16)$$

$$p_z = a_2 c_2 - d_4 c_{23} - d_6 (c_5 c_{23} + c_4 s_5 s_{23}) + d_5 s_4 s_{23} - d_1. \quad (17)$$

La forma de p_x y p_y muestra que q_1 solo gira el conjunto alrededor del eje vertical, sin modificar p_z . La matriz de rotación $R(q)$ tiene los elementos:

$$r_{11} = -c_1 c_{23} (c_4 c_5 c_6 - s_4 s_6) - c_1 c_6 s_5 s_{23} + s_1 (c_4 s_6 + c_5 c_6 s_4), \quad (18)$$

$$r_{12} = -c_1 c_{23} (c_4 c_5 s_6 + c_6 s_4) - c_1 s_5 s_6 s_{23} - s_1 (c_4 c_6 - c_5 s_4 s_6), \quad (19)$$

$$r_{13} = -c_1 c_4 c_{23} s_5 + c_1 c_5 s_{23} + s_1 s_4 s_5, \quad (20)$$

$$r_{21} = c_1 (c_4 s_6 + c_5 c_6 s_4) + s_1 c_{23} (c_4 c_5 c_6 - s_4 s_6) + s_1 c_6 s_5 s_{23}, \quad (21)$$

$$r_{22} = -c_1 (c_4 c_6 - c_5 s_4 s_6) + s_1 c_{23} (c_4 c_5 s_6 + c_6 s_4) + s_1 s_5 s_6 s_{23}, \quad (22)$$

$$r_{23} = c_1 s_4 s_5 + c_4 c_{23} s_1 s_5 - c_5 s_1 s_{23}, \quad (23)$$

$$r_{31} = s_{23} (c_4 c_5 c_6 - s_4 s_6) - c_6 c_{23} s_5, \quad (24)$$

$$r_{32} = s_{23} (c_4 c_5 s_6 + c_6 s_4) - c_{23} s_5 s_6, \quad (25)$$

$$r_{33} = c_4 s_5 s_{23} + c_5 c_{23}. \quad (26)$$

Como representación interpretable de la orientación se utiliza el cuaternión unitario (x, y, z, w) , calculado a partir de $R(q)$ eligiendo, en cada caso, la fórmula numéricamente estable según el mayor elemento de la diagonal. Por ejemplo, para la configuración

$$q = (0.2884, 0.9807, 0.5695, -0.5265, 0.5986, -1.2057) \text{ rad} \quad (27)$$

se obtiene:

$$p = (0.4151, -0.1712, 0.9606) \text{ m}, \quad (28)$$

$$\mathbf{q}_{xyzw} = (0.1518, 0.0310, -0.7611, 0.6299). \quad (29)$$

IV-D. Comprobaciones matemáticas

Para la configuración anterior, la matriz de rotación calculada cumple:

$$\max |R^T R - I| = 2.2 \times 10^{-16}, \quad (30)$$

$$\det(R) = 1.000000. \quad (31)$$

El mismo resultado se obtuvo en las tres configuraciones de la validación de la cinemática directa. Además, las expresiones simbólicas de $p(q)$ y $R(q)$ se evaluaron en configuraciones aleatorias y se compararon con la implementación numérica del nodo `fk_node`, con una diferencia máxima de 3.3×10^{-16} , del orden de la precisión de máquina.

El Jacobiano posicional se calcula en el proyecto de forma geométrica, a partir de los marcos intermedios T_{base}^i obtenidos durante la cinemática directa. Para cada articulación revoluita [4], [5]:

$$J_{v_i} = z_{i-1} \times (o_6 - o_{i-1}), \quad (32)$$

donde z_{i-1} es la tercera columna de R_{base}^{i-1} y o_{i-1} es la cuarta columna de T_{base}^{i-1} , ambos expresados en `base_link`.

V-A. Jacobiano utilizado en el proyecto

De forma equivalente, $J_v = \partial p / \partial q$. Derivando las expresiones de la sección anterior, y dado que h no depende de q_2 ni de q_3 , se obtiene:

$$J_v = [J_1 \quad J_2 \quad J_3 \quad J_4 \quad J_5 \quad J_6], \quad (33)$$

$$\begin{aligned} J_1 &= \begin{bmatrix} p_y \\ -p_x \\ 0 \end{bmatrix}, & J_2 &= \begin{bmatrix} c_1 \rho_2 \\ -s_1 \rho_2 \\ z_2 \end{bmatrix}, \\ J_3 &= \begin{bmatrix} c_1 \rho_3 \\ -s_1 \rho_3 \\ z_3 \end{bmatrix}, & J_6 &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \\ J_4 &= \begin{bmatrix} c_1 \rho_4 + s_1 h_4 \\ -s_1 \rho_4 + c_1 h_4 \\ z_4 \end{bmatrix}, & J_5 &= \begin{bmatrix} c_1 \rho_5 + s_1 h_5 \\ -s_1 \rho_5 + c_1 h_5 \\ z_5 \end{bmatrix}, \end{aligned} \quad (34)$$

donde $\rho_i = \partial \rho / \partial q_i$, $h_i = \partial h / \partial q_i$ y $z_i = \partial p_z / \partial q_i$:

$$\rho_2 = a_2 c_2 - d_4 c_{23} + d_5 s_4 s_{23} - d_6 (c_4 s_5 s_{23} + c_5 c_{23}), \quad (35)$$

$$\rho_3 = -(\rho_2 - a_2 c_2), \quad (36)$$

$$\rho_4 = -c_{23} (d_5 c_4 + d_6 s_4 s_5), \quad (37)$$

$$\rho_5 = d_6 (s_5 s_{23} + c_4 c_5 c_{23}), \quad (38)$$

$$h_4 = d_5 s_4 - d_6 c_4 s_5, \quad h_5 = -d_6 s_4 c_5, \quad (39)$$

$$z_2 = -a_2 s_2 + d_4 s_{23} + d_5 s_4 c_{23} - d_6 (c_4 s_5 c_{23} - c_5 s_{23}), \quad (40)$$

$$z_3 = -(z_2 + a_2 s_2), \quad (41)$$

$$z_4 = s_{23} (d_5 c_4 + d_6 s_4 s_5), \quad (42)$$

$$z_5 = d_6 (s_5 c_{23} - c_4 c_5 s_{23}). \quad (43)$$

La sexta columna es nula porque el efector se encuentra sobre el eje de `joint_6`: esa articulación modifica la orientación, pero no la posición.

La implementación geométrica se verificó contra un Jacobiano numérico por diferencias centrales ($\Delta q = 10^{-6}$ rad). Para la configuración de ejemplo de la cinemática directa, la diferencia máxima entre ambos fue de 8.8×10^{-11} .

V-B. Consideración de singularidades

Cuando J_v pierde rango, existen direcciones cartesianas que el efector no puede seguir de forma instantánea, y la pseudo-inversa produce pasos articulares muy grandes o poco útiles. El caso más evidente es $q = 0$: con el brazo completamente extendido en vertical, el Jacobiano posicional tiene rango 1 (valores singulares 1.03, 0, 0). En cambio, en la postura home de Kinova sus valores singulares son 0.598, 0.474 y 0.288, por lo que el método iterativo se inicia lejos de la singularidad. Algo similar ocurre cerca del límite del espacio de trabajo, donde el brazo vuelve a quedar casi extendido; este comportamiento se observa en el caso no convergente de la validación de la cinemática inversa.

VI. CINEMÁTICA INVERSA

VI-A. Definición del problema

El objetivo es encontrar $q^* \in \mathbb{R}^6$ tal que la posición del efector alcance un punto deseado:

$$p(q^*) \approx p_d = [x_d \ y_d \ z_d]^T. \quad (44)$$

Se resuelve únicamente la posición; la orientación se obtiene en la cinemática directa. Como existen seis articulaciones para tres ecuaciones de posición, el problema es redundante y admite infinitas soluciones; la configuración alcanzada depende de la semilla inicial.

VI-B. Objetivo recibido mediante ROS 2

El nodo `ik_node` se suscribe al tópico `/target` con mensajes `geometry_msgs/msg/Point`. Un objetivo se envía desde terminal con:

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x:_0.40,_y:_0.20,_z:_0.50}" -t 3
```

Se utiliza `-t 3` en lugar de `-once` porque, en las pruebas, un único mensaje podía enviarse antes de establecerse la conexión con el suscriptor y perderse.

VI-C. Método numérico iterativo

En cada iteración se calcula el error de posición y se actualiza la configuración con la pseudo-inversa del Jacobiano (33):

$$e_k = p_d - p(q_k), \quad (45)$$

$$\Delta q_k = \alpha J_v^+(q_k) e_k, \quad (46)$$

$$q_{k+1} = \text{clip}(q_k + \Delta q_k, q_{\min}, q_{\max}). \quad (47)$$

La pseudo-inversa se calcula numéricamente con `numpy.linalg.pinv`, que la obtiene mediante descomposición en valores singulares. Como J_v es de 3×6 y de rango completo fuera de singularidades, equivale a $J^+ = J^T(JJ^T)^{-1}$ y entrega el paso Δq de norma mínima. Adicionalmente, si $\|\Delta q_k\|_2 > 0.2$ rad, el paso se escala a esa norma: lejos de la solución la linealización del Jacobiano es poco precisa y un paso grande puede hacer oscilar el método.

VI-D. Configuración inicial

En la primera solicitud, el algoritmo parte de la postura home de Kinova:

$$\begin{aligned} q_0 &= (0^\circ, 15^\circ, -130^\circ, 0^\circ, 55^\circ, 90^\circ) \\ &= (0, 0.2618, -2.2689, 0, 0.9599, 1.5708) \text{ rad.} \end{aligned} \quad (48)$$

No se usa $q = 0$ porque es una configuración singular. En las solicitudes siguientes, la semilla es la última solución convergente, de modo que el brazo se desplaza con continuidad entre objetivos.

VI-E. Articulaciones utilizadas

Intervienen las seis articulaciones. Sin embargo, la columna de q_6 en J_v es nula, por lo que la pseudo-inversa nunca la modifica: q_6 conserva el valor de la semilla (1.5708 rad). Esto no afecta la solución, ya que q_6 no influye en la posición del efector.

VI-F. Límites articulares

Los límites provienen del URDF de `kortex_description`. Las articulaciones 1, 4 y 6 son continuas (sin límite), por lo que se acotaron a $[-\pi, \pi]$ para obtener soluciones con un rango definido.

Cuadro IV
LÍMITES ARTICULARES UTILIZADOS

Joint	Mín. [rad]	Máx. [rad]	Tipo
q_1	$-\pi$	π	continua
q_2	-2.24	2.24	revoluta
q_3	-2.57	2.57	revoluta
q_4	$-\pi$	π	continua
q_5	-2.09	2.09	revoluta
q_6	$-\pi$	π	continua

VI-G. Criterio de convergencia

El algoritmo converge cuando el error euclidiano de posición cumple $e_p = \|p_d - p(q_k)\|_2 < \varepsilon$ antes del máximo de iteraciones. Los parámetros utilizados son:

Cuadro V
PARÁMETROS DEL MÉTODO ITERATIVO

Parámetro	Valor
Factor de actualización α	0.5
Tolerancia ε	1×10^{-4} m
Máximo de iteraciones	500
Paso máximo $\ \Delta q\ _2$	0.2 rad

Si no se alcanza la tolerancia, el nodo informa NOT CONVERGED y conserva la configuración anterior, en lugar de publicar una solución que no alcanza el objetivo.

VI-H. Publicación de la solución articular

La solución convergente se publica en `/joint_states` como `sensor_msgs/msg/JointState`, con los nombres `joint_1` a `joint_6` en el orden del URDF. La publicación se realiza de forma periódica a 10 Hz, de modo que `robot_state_publisher` y `RViz2` muestran siempre la configuración vigente.

VI-I. Verificación mediante cinemática directa

Para cada solución se evalúa $p_f = p(q^*)$ con la misma cinemática directa y se reporta $\|p_d - p_f\|_2$. De forma independiente, el nodo `fk_node` recibe la solución publicada en `/joint_states` y vuelve a calcular la pose, lo que permite comprobar en consola que el efector alcanzó el objetivo. Los resultados se presentan en la validación de la cinemática inversa.

VII. IMPLEMENTACIÓN EN PYTHON Y ROS 2

VII-A. Herramientas

La implementación se desarrolló y probó con:

- Ubuntu 24.04 LTS;
- ROS 2 Jazzy, con el middleware CycloneDDS (`rmw_cyclonedds_cpp`);
- Python 3.12 y NumPy;
- `RViz2`, `robot_state_publisher` y `joint_state_publisher_gui`;
- `SymPy`, empleado fuera de los nodos para el desarrollo simbólico de la Sección IV.

Toda la matemática se concentra en el módulo `kinematics.py`, que contiene la tabla DH, la transformación fija T_{base}^0 , las funciones de cinemática directa, el Jacobiano posicional, la conversión de matriz de rotación a cuaternión y los límites articulares. Los dos nodos importan este módulo, de modo que la cinemática directa y la inversa comparten exactamente el mismo modelo.

VII-B. Nodo de cinemática directa

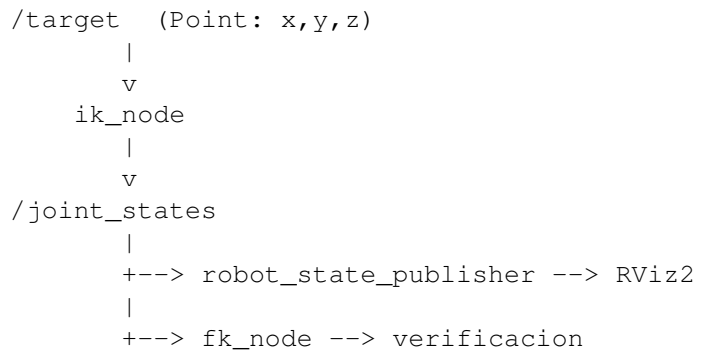
El nodo `fk_node`:

1. se suscribe a `/joint_states` (`sensor_msgs/msg/JointState`);
2. ordena las articulaciones por nombre (`joint_1` a `joint_6`), sin asumir el orden del mensaje;
3. evalúa las matrices DH y calcula T_{base}^6 según (4);
4. extrae la posición (x, y, z) y la matriz de rotación R ;
5. convierte R en cuaternión (x, y, z, w) ;
6. muestra q , la posición, el cuaternión y R en consola cada vez que cambia la configuración;
7. publica la pose en `/fk_pose` (`geometry_msgs/msg/PoseStamped`).

El nodo no consulta TF: la pose se calcula únicamente con el modelo DH del grupo. TF se utilizó solo como referencia para la validación.

VII-C. Nodo de cinemática inversa

El nodo `ik_node` se suscribe a `/target` (`geometry_msgs/msg/Point`), resuelve la cinemática inversa de posición según la Sección VI y publica la solución en `/joint_states`. El flujo de información es:



Por cada objetivo recibido, el nodo reporta en consola el objetivo (x_d, y_d, z_d) , la configuración inicial q_0 , la solución q^* , el número de iteraciones, la posición alcanzada $p(q^*)$, el error final y el estado `CONVERGED` o `NOT CONVERGED`. Un ejemplo de salida es:

```

--- IK ---
target p_d [m]   = [0.4 0.2 0.5]
q0 [rad]         = [0. 0.2618 -2.2689 0. 0.9599
1.5708]
q* [rad]         = [-0.4198 0.1541 -2.2085 0.1294
0.9999 1.5708]
iterations       = 13
p(q*) [m]        = [0.4 0.1999 0.5]
error ||e|| [m]  = 0.000064
status           = CONVERGED

```

VII-D. Interacción con Joint State Publisher GUI

El `Joint State Publisher GUI` y `ik_node` publican en el mismo tópico `/joint_states`. Si ambos están activos, la postura del GUI sobrescribe la solución de la cinemática inversa. Para evitar el conflicto se emplean dos launch files:

- `display.launch.py`: modelo oficial, `RViz2` y `Joint State Publisher GUI`, para las pruebas de cinemática directa.
- `ik_demo.launch.py`: el mismo modelo y `RViz2`, sin el GUI, junto con `ik_node` y `fk_node`, para las pruebas de cinemática inversa.

De este modo, durante las pruebas de cinemática inversa `ik_node` es el único publicador de `/joint_states`.

VIII. WORKSPACE ROS 2

VIII-A. Estructura

```

grupo_09_kinova_gen3_6dof_ws/
|-- src/
|   |-- grupo09_kinova_gen3_bringup/
|   |   |-- launch/
|   |   |   |-- display.launch.py
|   |   |   |-- ik_demo.launch.py
|   |   |-- CMakeLists.txt
|   |   |-- package.xml
|   |
|   |-- grupo09_kinova_gen3_kinematics/

```

```
|
|   |-- grupo09_kinova_gen3_kinematics/
|   |   |-- kinematics.py
|   |   |-- fk_node.py
|   |   |-- ik_node.py
|   |   |-- resource/
|   |   |-- package.xml
|   |   |-- setup.py
|   |   |-- setup.cfg
|   |
|   |-- docs/
|   |   |-- figura_frames.png / .pdf
|   |   |-- capturas/
|   |-- dependencias.repos
|   |-- instalar.sh
|   |-- entorno.sh
|   |-- README.md
|   |-- requirements.txt
|   |-- .gitignore
```

El paquete oficial `kortex_description` no se incluye en el repositorio: se descarga en `src/ros2_kortex` mediante `dependencias.repos`, fijado a un commit conocido.

VIII-B. Compilación

Desde la raíz del workspace:

```
source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

rosdep install --from-paths src --ignore-src -r -y

colcon build --symlink-install
source install/setup.bash
```

En cada terminal nueva basta con ejecutar `source entorno.sh`, que carga ROS 2 Jazzy, CycloneDDS y el workspace.

VIII-C. Ejecución de la cinemática directa

Terminal 1:

```
ros2 launch grupo09_kinova_gen3_bringup display.launch.py
```

Terminal 2:

```
ros2 run grupo09_kinova_gen3_kinematics fk_node
```

VIII-D. Ejecución de la cinemática inversa

Terminal 1 (incluye `ik_node` y `fk_node`):

```
ros2 launch grupo09_kinova_gen3_bringup ik_demo.launch.py
```

Terminal 2:

```
ros2 topic pub /target geometry_msgs/msg/Point \
"{x:_0.40,_y:_0.20,_z:_0.50}" -t 3
```

IX. VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Como referencia se utilizó la transformación `base_link` → `end_effector_link` que calcula TF a partir del URDF oficial, obtenida con:

```
ros2 run tf2_ros tf2_echo base_link
end_effector_link
```

Para cada caso, la misma configuración q publicada en `/joint_states` se evaluó con `fk_node` y con TF. Se probaron la configuración cero, una configuración general con las seis articulaciones distintas de cero y una configuración con el brazo flexionado hacia atrás.

Cuadro VI
VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Caso	Magnitud	Valor
1	q [rad]	(0, 0, 0, 0, 0, 0)
	p_{calc} [m]	(0.0000, 0.0014, 1.1766)
	p_{ROS} [m]	(0.000, 0.001, 1.177)
	e_p / e_o	5.1×10^{-4} m / 0 rad
2	q [rad]	(0.288, 0.981, 0.570, -0.527, 0.599, -1.206)
	p_{calc} [m]	(0.4151, -0.1712, 0.9606)
	p_{ROS} [m]	(0.415, -0.171, 0.961)
	e_p / e_o	4.7×10^{-4} m / 5.1×10^{-4} rad
3	q [rad]	(0, -1.175, 0, 2.327, 0, 0)
	p_{calc} [m]	(-0.8227, 0.0008, 0.6292)
	p_{ROS} [m]	(-0.823, 0.001, 0.629)
	e_p / e_o	4.5×10^{-4} m / 1.1×10^{-3} rad

El error de posición se calculó como:

$$e_p = \|p_{calc} - p_{ROS}\|_2, \quad (49)$$

y el error de orientación como el ángulo entre los cuaterniones calculado y de referencia, $e_o = 2 \arccos |\mathbf{q}_{calc} \cdot \mathbf{q}_{ROS}|$.

En los tres casos el error es menor a 0.5 mm en posición y a 1.1×10^{-3} rad en orientación. Estos valores corresponden al redondeo de `tf2_echo`, que muestra solo tres decimales: las matrices de rotación calculadas coinciden con las de TF en todos los dígitos mostrados. Además, en el caso 3 se comprobó la posición de forma independiente desde RViz2 (RobotModel → tool_frame → Position).

IX-A. Pruebas por transformaciones parciales

La cadena se validó también de forma incremental, comparando en $q = 0$ el eje z_{i-1} de cada producto parcial T_{base}^{i-1} con el eje de la articulación i en el URDF, expresado en `base_link`. La Tabla VII muestra que coinciden en los seis casos, y que el origen del marco $\{1\}$ queda a la altura del eje del hombro (0.2848 m). Esta comparación permitió confirmar el sentido de cada eje antes de evaluar la transformación completa.

Cuadro VII
VALIDACIÓN INCREMENTAL EN $q = 0$

Producto	z_{i-1} (DH)	Eje URDF	Coincide
T_{base}^0	(0, 0, -1)	joint_1: (0, 0, -1)	Sí
$T_{base}^0 A_1^1$	(0, 1, 0)	joint_2: (0, 1, 0)	Sí
$\dots A_1^2$	(0, -1, 0)	joint_3: (0, -1, 0)	Sí
$\dots A_2^3$	(0, 0, -1)	joint_4: (0, 0, -1)	Sí
$\dots A_3^4$	(0, -1, 0)	joint_5: (0, -1, 0)	Sí
$\dots A_4^5$	(0, 0, -1)	joint_6: (0, 0, -1)	Sí

X. VALIDACIÓN DE LA CINEMÁTICA INVERSA

Se enviaron tres objetivos alcanzables mediante `/target`, en forma consecutiva: el primero parte de la postura home y cada uno de los siguientes parte de la solución anterior (q_1^* y q_2^*). Adicionalmente, se envió un objetivo fuera del espacio de trabajo para documentar el comportamiento sin convergencia. En todos los casos, la posición alcanzada se comprobó con `fk_node` y con TF.

Cuadro VIII
PRUEBAS DE CINEMÁTICA INVERSA

Caso	p_d [m]	q_0	Iter.	Error [m]	Estado
1	(0.40, 0.20, 0.50)	home	13	6.4×10^{-5}	convergió
2	(0.30, -0.25, 0.60)	q_1^*	17	5.3×10^{-5}	convergió
3	(0.50, 0.00, 0.30)	q_2^*	15	8.5×10^{-5}	convergió
4	(1.20, 0.00, 0.30)	q_3^*	500	0.309	no conv.

Cuadro IX
SOLUCIONES ARTICULARES OBTENIDAS

Caso	q^* [rad]
1	(-0.420, 0.154, -2.209, 0.129, 1.000, 1.571)
2	(0.601, -0.051, -2.191, -0.261, 1.064, 1.571)
3	(-0.012, 0.550, -2.184, -0.056, 0.888, 1.571)

Los tres objetivos alcanzables convergieron en 13 a 17 iteraciones, con errores finales por debajo de la tolerancia $\varepsilon = 10^{-4}$ m. TF confirmó las posiciones (0.400, 0.200, 0.500), (0.300, -0.250, 0.600) y (0.500, 0.000, 0.300) m. En el caso 4, el objetivo se encuentra a unos 1.2 m del hombro, mientras que el alcance del brazo es de unos 0.89 m; el algoritmo agotó las 500 iteraciones con el efector en (0.891, 0.000, 0.296) m, informó NOT CONVERGED y conservó la configuración anterior.

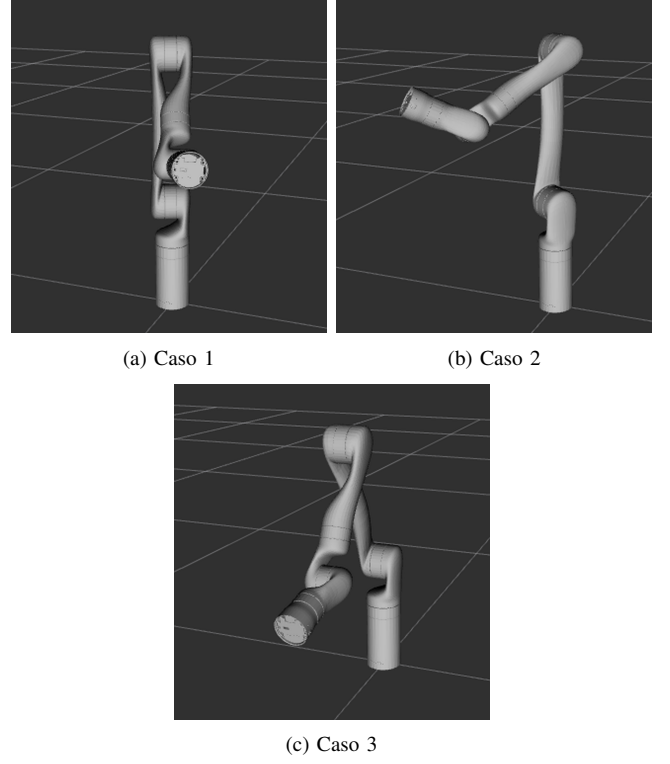


Figura 3. Configuraciones obtenidas por la cinemática inversa en RViz2 para los objetivos (a) (0.40, 0.20, 0.50) m, (b) (0.30, -0.25, 0.60) m y (c) (0.50, 0.00, 0.30) m.

XI. REPOSITORIO GITHUB Y REPRODUCIBILIDAD

XI-A. Enlace

- Repositorio: https://github.com/migueltorrezv/grupo_09_kinova_gen3_6dof_ws
- Rama de entrega: main
- Commit evaluado: 8a77048

XI-B. Contenido del repositorio

El repositorio contiene el `README.md`, los dos paquetes ROS 2 del grupo (`grupo09_kinova_gen3_bringup` y `grupo09_kinova_gen3_kinematics`) con sus `launch files`, `package.xml`, `setup.py`, `setup.cfg` y `CMakeLists.txt`, además de `dependencias.repos`, `instalar.sh`, `entorno.sh`, `requirements.txt`, `.gitignore` y la carpeta `docs/` con la figura de marcos y las capturas de validación.

Mediante `.gitignore` se excluyen `build/`, `install/`, `log/`, `__pycache__/`, `*.pyc`, `.vscode/` e `.idea/`, así como `src/ros2_kortex/`, que es código del fabricante.

XI-C. README

El `README.md` incluye el objetivo, los autores, el software requerido, las instrucciones de clonación, instalación, compilación y ejecución, un ejemplo concreto de prueba, la tabla DH, los parámetros de la cinemática inversa, los tópicos utilizados, la estructura del repositorio y las consideraciones conocidas.

XI-D. Procedimiento de reproducción

```
git clone https://github.com/migueltorrezv/
grupo_09_kinova_gen3_6dof_ws.git ~/
grupo_09_kinova_gen3_6dof_ws
cd ~/grupo_09_kinova_gen3_6dof_ws

chmod +x instalar.sh
./instalar.sh

source entorno.sh
ros2 launch grupo09_kinova_gen3_bringup display.
launch.py
```

y posteriormente, en otra terminal:

```
source entorno.sh
ros2 run grupo09_kinova_gen3_kinematics fk_node
```

instalar.sh instala las dependencias del sistema, descarga el modelo oficial con `vcs import`, resuelve dependencias con `rosdep` y compila el workspace. El procedimiento se verificó clonando el repositorio en una carpeta nueva y ejecutando la instalación completa desde cero.

XI-E. Dependencias externas reproducibles

El modelo oficial se fija al commit utilizado durante el desarrollo:

```
repositories:
  ros2_kortex:
    type: git
    url: https://github.com/Kinovarobotics/
    ros2_kortex.git
    version: 462
    dab9aa4732d733be55e1846530dd920c7c7d3
```

XII. RESULTADOS Y DISCUSIÓN

Precisión del modelo. La cinemática directa reproduce el modelo oficial: las diferencias con TF son del orden del redondeo de la herramienta de comparación, y las expresiones simbólicas coinciden con la implementación numérica hasta la precisión de máquina. Esto indica que la inclusión de los desplazamientos laterales pequeños (d_2 , d_3 , d_5) fue necesaria para obtener una coincidencia exacta; despreciarlos habría introducido errores de algunos milímetros.

Asignación de marcos. La principal dificultad fue que el eje de `joint_1` apunta en $-z$ en el URDF. Si el marco $\{0\}$ se hubiera tomado igual a `base_link`, el signo de q_1 quedaría invertido respecto de ROS. Se resolvió con la transformación fija $R_x(\pi)$. Como consecuencia, un valor positivo de q_1 gira el brazo en sentido horario visto desde arriba, y el efector se desplaza hacia y negativo; este comportamiento coincide con TF y no constituye un error de signo. Del mismo modo, el modelo admite configuraciones dentro de los límites articulares en las que el efector queda por debajo de la base ($z < 0$); son válidas matemáticamente, pero el robot real colisionaría con su superficie de montaje, que el URDF no modela.

Sensibilidad a la semilla. Al ser un problema redundante (seis articulaciones para tres ecuaciones), la configuración final depende de q_0 . Utilizar la solución anterior como semilla produce movimientos continuos entre objetivos, pero implica que el mismo objetivo puede alcanzarse con configuraciones

distintas según el historial. Por la misma razón, q_6 permaneció constante en todas las pruebas: su columna del Jacobiano posicional es nula.

Singularidades. En $q = 0$ el Jacobiano posicional tiene rango 1, por lo que el método no puede iniciarse desde esa configuración; por ello se eligió la postura home. En el objetivo inalcanzable, el algoritmo llevó el brazo hasta su máxima extensión (≈ 0.89 m) y quedó estancado cerca de una singularidad, sin poder reducir el error. La detección de no convergencia evitó publicar una solución incorrecta. Un método de mínimos cuadrados amortiguados mejoraría el comportamiento en estas zonas.

Integración en ROS 2. Se identificaron tres problemas prácticos: el conflicto entre el GUI y `ik_node` al publicar ambos en `/joint_states`, resuelto con un launch sin GUI; la falta de respuesta del GUI bajo Wayland en Ubuntu 24.04, resuelta forzando X11 con `QT_QPA_PLATFORM=xcb`; y la pérdida ocasional de mensajes enviados con `-once`, resuelta publicando el objetivo varias veces.

XIII. CONCLUSIONES

- Se obtuvo una tabla Denavit–Hartenberg estándar del Kinova Gen3 6-DOF, complementada con una transformación fija $R_x(\pi)$, cuya cinemática directa coincide con el modelo oficial de ROS con errores menores a 0.5 mm, limitados por el redondeo de la referencia.
- Hacer coincidir cada z_{i-1} con el eje y el sentido de la articulación en el URDF simplificó la tabla, dejando todos los ángulos como q_i más un offset constante; la validación incremental de los ejes permitió detectar los cambios de sentido antes de evaluar la cadena completa.
- El método iterativo con pseudoinversa del Jacobiano, con paso acotado, límites articulares y semilla fuera de singularidad, resolvió los tres objetivos alcanzables en 13 a 17 iteraciones con errores menores a 0.1 mm, y detectó correctamente un objetivo fuera del espacio de trabajo.
- La redundancia del robot frente a una tarea de solo posición hace que la solución dependa de la semilla y deja q_6 sin efecto sobre la posición; resolver también la orientación permitiría aprovechar los seis grados de libertad.
- La separación del modelo matemático en un módulo común y el uso de launch files distintos para cinemática directa e inversa evitaron inconsistencias entre nodos y conflictos en `/joint_states`, y el repositorio con dependencias fijadas permitió reproducir el proyecto desde cero.

REFERENCIAS

- [1] Kinova Robotics, “Gen3 robots,” <https://www.kinovarobotics.com/product/gen3-robots>, consultado en octubre de 2026.
- [2] Kinova Robotics, “ros2_kortex: ROS 2 driver for the Gen3 Kinova robot arm,” GitHub, https://github.com/Kinovarobotics/ros2_kortex, commit 462dab9.
- [3] Open Robotics, “ROS 2 Documentation: Jazzy,” <https://docs.ros.org/en/jazzy/>, consultado en octubre de 2026.
- [4] M. W. Spong, S. Hutchinson y M. Vidyasagar, *Robot Modeling and Control*, 2.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2020.
- [5] B. Siciliano, L. Sciacicco, L. Villani y G. Oriolo, *Robotics: Modelling, Planning and Control*. Londres, Reino Unido: Springer, 2009.